bcast/flat

An AgentMPI collective scaling analysis

Abstract

The flat broadcast is the algorithm almost every agent framework uses without naming it: the root sends the artifact to each of the other $p-1$ ranks in a loop. Across 21 measured sizes from $p=2$ to $p=32$ the implementation logged exactly $p-1$ messages, matching its closed form at every size and reporting that count exactly; there is no remainder path to get wrong, because the loop does not care whether p is a power of two. The sweep's real contribution is to separate two costs the phrase "$p-1$ rounds" conflates. Flat broadcast has a network depth of one — every leaf receives directly from the root — and a root-side issue cost of $p-1$, and only the second grows. Measured, that issue cost is 0.90 ms per send at $p=32$ against 10.4 ms at $p=2$, so the 31 sends that the cost model charges as 31 rounds are a 28 ms burst in one rank's lane rather than 31 round trips. The single most important thing to take away is that a broadcast never invokes an agent: all three broadcast algorithms carried exactly *one* distinct content digest at every measured size, from one hop to thirty-one, so the flat algorithm's fan-out costs fabric writes and root serialisation and nothing else — no operator applications, no re-encodings, and a fold depth of zero at all 21 sizes.

1 What this family is

The `bcast` collective under the `flat` algorithm, measured across 21 runs at 13 distinct process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.012–0.093 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

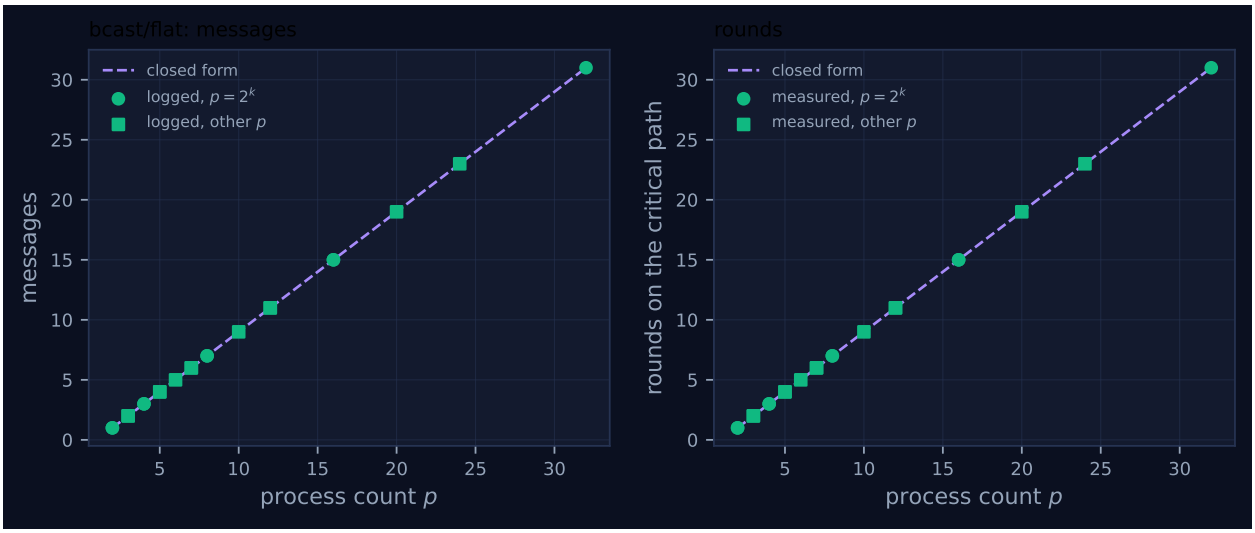

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.012	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.012	✓	✓
3	`mb-free`	2	2	2	2	2	0.012	✓	✓
3	`mb-smoke`	2	2	2	2	2	0.014	✓	✓
4	`mb-free`	3	3	3	3	3	0.013	✓	✓
4	`mb-smoke`	3	3	3	3	3	0.015	✓	✓
5	`mb-free`	4	4	4	4	4	0.014	✓	✓
5	`mb-smoke`	4	4	4	4	4	0.014	✓	✓
6	`mb-free`	5	5	5	5	5	0.017	✓	✓
7	`mb-free`	6	6	6	6	6	0.018	✓	✓
7	`mb-smoke`	6	6	6	6	6	0.029	✓	✓
8	`mb-free`	7	7	7	7	7	0.015	✓	✓
8	`mb-smoke`	7	7	7	7	7	0.017	✓	✓
10	`mb-free`	9	9	9	9	9	0.023	✓	✓
12	`mb-free`	11	11	11	11	11	0.023	✓	✓
12	`mb-smoke`	11	11	11	11	11	0.021	✓	✓
16	`mb-free`	15	15	15	15	15	0.029	✓	✓
16	`mb-smoke`	15	15	15	15	15	0.026	✓	✓
20	`mb-free`	19	19	19	19	19	0.040	✓	✓
24	`mb-free`	23	23	23	23	23	0.065	✓	✓
32	`mb-free`	31	31	31	31	31	0.093	✓	✓

4 How the algorithm scales

The implementation is four lines. The root iterates over all destinations and calls `comm._csend` once per non-root rank; every other rank performs a single `comm._crecv` from the root. That gives $p - 1$ messages, and because the payload is identical for every destination it gives $(p-1)n$ tokens of volume,

which is exactly the closed form `FORMULAS[("bcast","flat")]` records as $(p-1,\ p-1,\ (p-1)n,\ 0)$. The fourth term, fold depth, is zero because a broadcast applies no operator; the sweep confirms it at 21 of 21 sizes.

The first entry of that tuple is the one worth unpacking, because it is not a latency depth. The implementation records `rounds` asymmetrically: $p-1$ at the root and 1 at every leaf. The family analysis takes the maximum over participants, so the "rounds" column of the table is the root's figure. What it counts is how many sends one rank has to issue sequentially, not how many hops separate the root from the furthest rank — that distance is one, for every p. The distinction is invisible at $p = 2$, where the two readings coincide, and it is the whole story at $p = 32$.

The trace shows the issue cost directly. In `mb-free__coll-bcast-flat-8` the root's seven `msg.send` events are stamped at 5.10, 5.24, 5.33, 5.41, 5.49, 5.57 and 5.64 ms after the first event of the run: seven sends about 90 μs apart, and the root's own `coll.bcast` record closes at 1.8 ms of blocking. Dividing each size's maximum per-rank blocking time by its round count gives a per-send cost that *falls* monotonically with p — 10.4 ms at $p = 2$, 1.49 ms at $p = 8$, 1.25 ms at $p = 16$, 0.90 ms at $p = 32$ — because at small p the figure is dominated by the fixed cost of reaching the collective at all, and at large p it converges on the marginal cost of one SQLite write inside a tight loop. There is no size at which the $p-1$ rounds behave like $p-1$ serialised network traversals.

That is why the left panel of Figure 1 is a straight line through every marker and the right panel is the same straight line: for this algorithm the message count and the model's round count are the same quantity, and both are exactly $p-1$. Nothing steps, because there is nothing in the algorithm that could step. Comparing the two panels of this figure against `bcast/binomial`'s is the cleanest statement of what tree broadcast buys: the same left panel, and a logarithmic staircase on the right.

The viewer screenshot for `mb-free__coll-bcast-flat-32` shows the shape the numbers imply. Rank 0's lane carries a dense, bright cluster of message glyphs between roughly 0.015 and 0.035 s and nothing afterwards; the other thirty-one lanes each carry one or two ticks scattered across the span, in an order that has nothing to do with the order the root sent in. At $p = 8$ the leaves record their `coll.bcast` in the order $7, 6, 1, 4, 2, 3, 5$ while the root sent in the order $1, \ldots, 7$, and the per-message `latency_s` field climbs from 0.001 to 0.009 s across the seven deliveries. Neither the reordering nor the climb is a property of the algorithm: they are the operating system waking thirty-two threads that are all polling the same SQLite file. What the picture does establish is the absence of any structure between the leaves — no lane waits for another lane, which is precisely the claim "network depth one" makes.

5 Powers of two and everything else

This family has no remainder path. The root's loop is `for d in range(p)` with a single `if d != root` guard, and the leaves' behaviour does not depend on p at all, so the code executed at $p = 7$ is the code executed at $p = 8$ with one fewer iteration. The figure marks eight non-power-of-two sizes $(3, 5, 6, 7, 10, 12, 20, 24)$ as squares and five powers of two as circles, and every marker in both panels lies on the closed-form line: 2 messages at $p = 3$, 9 at $p = 10$, 19 at $p = 20$, 23 at $p = 24$. There are no red crosses, and the self-reported count equals the logged count at all 21 sizes, so the accounting invariant that caught the recursive-doubling allreduce under-count holds here without qualification.

The token accounting is exact as well, which is worth recording because it is not true of every member of the `reduce` family. Summing the `tokens` field of the `msg.send` events gives $p-1$ at every size, identical to the $p-1$ the collective reports as `tokens_sent`. The flat broadcast sends the

caller's payload with no wrapper, so the transport's view and the algorithm's view of the volume cannot diverge.

Eight of the thirteen distinct process counts were measured twice, once under `mb-free` and once under `mb-smoke`. At all eight the logged count, the self-reported count, the round count, the token volume and the participant count are identical; the run wall times differ by factors between 1.01 and 1.62. Two independent campaigns agreeing on every count and disagreeing on every duration is the correct summary of what this sweep can and cannot measure.

6 When to choose this algorithm

All three broadcast algorithms move $p - 1$ messages, so the choice among them is purely a choice of shape, and this is where the broadcast family differs fundamentally from the reduction family that mirrors it. A broadcast copies. Because every payload is content-addressed and forwarding moves the handle rather than the content, an interior rank cannot paraphrase what it relays — and the sweep shows this rather than asserting it. Across all thirteen sizes and all three broadcast algorithms, the set of distinct digests appearing in `msg.send` payloads has size exactly one: at $p = 32$, thirty-one messages carry the same digest under `flat`, under `binomial` and under `chain` alike. The corresponding `reduce` sweeps carry $p - 1$ *distinct* digests under `binomial` and `chain`, even though every rank contributed the identical value 1 under `SUM`. Depth is free of re-encoding for a broadcast in a way it can never be for a reduction, and the digest column is the direct evidence.

Given that, flat broadcast's only liability is the root, and it is a real one on two counts. It is a serialisation point: the root cannot proceed until it has issued $p - 1$ sends, which measured 28 ms of blocking at $p = 32$ and would be $p - 1$ fabric writes plus $p - 1$ token transfers in the cost model's terms. And it is a fan-out concentration: if a harness admits the payload into the receiving rank's context, flat delivers $p - 1$ concurrent admissions against binomial's staged ones. What flat is *not* is an agent cost. The cost model charges a broadcast no per-invocation α term at all — `cost.predict` computes `rounds * (fabric_s + n_tokens * beta_in_s_per_token)` for a broadcast and adds an operator term only for reductions — and the sweep confirms the premise, with zero `agent.call` events and `executors` recorded as `{"none": p}` at every one of the 21 runs.

So the honest crossover is narrower than the round counts suggest. At $n = 1000$ tokens the model predicts 3.97 s for flat at $p = 32$ against 0.64 s for binomial, a factor of 6.2; measured, the two are 28.0 ms and 44.8 ms of maximum per-rank blocking, with flat *faster*. The discrepancy is not a defect in either number: the model charges one prompt-processing term per round, and this sweep never admitted a token into any context (zero admitted tokens across 1,110 `msg.recv` events in the six sweeps), so the term the model is dominated by is exactly the term the measurement sets to zero. Flat is the right choice when p is small, when the root has nothing else to do, and when the payload is not being admitted; binomial is the right choice when any of those fails, and `scatter_allgather` is the right choice when the artifact is too large for an interior rank's mailbox. Against `chain`, flat wins outright at every measured size — 28.0 ms against 1,481.8 ms of blocking at $p = 32$ — because chain pays $p - 1$ hops of fabric latency to buy a fan-out of one that flat's leaves never needed.

7 Threats to this reading

The most serious threat is that at the sizes where the wall times look most interesting, they are not measuring the collective. In `mb-free__coll-bcast-flat-32` the thirty-two `rank.init` events

span 3.3 to 90.1 ms while the whole run lasts 92.5 ms: the last rank started after the broadcast was effectively over. The run wall time of 0.093 s in the generated table is therefore a measurement of process launch, and the 73 ms spread between the first and last rank to record the collective is launch stagger, not synchronisation cost. The only wall-clock figure in this family I would defend is the maximum per-rank blocking time inside the collective (28.0 ms at $p = 32$), and even that is a handful of SQLite transactions.

Second, this family measures counts, not costs. The three quantities that agree with theory at 21 of 21 sizes — messages, rounds, fold depth — are integers produced by the implementation's own bookkeeping and by the fabric's transport log, and the agreement of those two independent sources is a genuine check. The cost model's α, β and γ coefficients are checked by nothing here: with `executors = {"none": 32}`, zero agent calls, zero busy time, an idle fraction of 1.0 and \$0.000 of cost, every term in $T(n) = \alpha + n\beta$ is multiplied by zero. Any claim in this document about what flat broadcast would cost a real agent population is a claim about the model, not about these runs.

Third, one broadcast configuration is exercised and several are not. Every run used `root = 0`; the rotation logic in the flat path is a single `if d != root` and is unlikely to be wrong, but it is untested by this archive. Every run used a scalar payload of one token, so the volume term $(p-1)n$ is validated only at $n = 1$ and the interaction between fan-out and context admission is not exercised at all, because no message in the sweep was admitted. And the two campaigns are two runs of the same code on the same machine minutes apart, so their agreement establishes determinism of the counts, not independence of the measurement apparatus.

Finally, the reading that flat's per-send cost falls with p rests on dividing per-rank blocking time by round count, a quantity I computed from `collective_wall_s` and `rounds` in `metrics.json` rather than one the harness reports. It is sensitive to the launch stagger described above: at $p = 32$ the root is among the earlier ranks to start, so its blocking time is not inflated by waiting for stragglers, but that is an accident of scheduling rather than something the harness guarantees.